Evolving Vision-Language Inference: An AlphaEvolve-Style Code Optimization System for Chart Question Answering

Anonymous Author(s)
Affiliation
Address
email

Abstract

We present an AlphaEvolve-style evolutionary code optimization system applied to Vision-Language Model (VLM) inference for chart understanding on the CharXiv benchmark. Starting from a naive baseline achieving 38.3% accuracy, we first manually optimize the inference pipeline through prompt engineering, structured output normalization, and image caching, reaching 53.9% accuracy with a 13× speed improvement. We then implement an evolutionary system that uses an LLM (GPT) as a mutation operator over a maintained population of candidate programs, automatically discovering prompt- and post-processing-level improvements (expanded Not-Applicable synonyms, question-type-aware gating, regex-based answer extraction, dynamic token budgets), reaching 60.9% accuracy. Finally, through systematic per-sample error analysis, we identify and fix five post-processing bugs (negative sign stripping, text tick mishandling, caret notation breakage, legend count deduplication, and trend synonym mismatch) that together yield a further 9.4 pp gain to 69.5%. A subsequent per-sample error analysis reveals that 13 of 39 remaining errors stem from the model hallucinating continuous legend (colorbar) values on charts that have none; a targeted one-shot colorbar verifier probe corrects all 13, pushing accuracy to **79.7%**—a 41.4 percentage point improvement over the baseline. A four-fold stability check on the 128 development samples yields 0.797 ± 0.031 for `best_accuracy`, suggesting that the verifier gains are robust across subsets (though this evaluates the same fixed program, not re-derived verifiers per fold). These results show that systematic error categorization and lightweight targeted verification can substantially outperform sophisticated inference techniques when the bottleneck is post-processing and hallucination rather than model perception.

1 Introduction

Chart question answering is a challenging multimodal task requiring a model to read visual artifacts (axes, legends, data lines) and produce exact short-form answers. The CharXiv benchmark [2] evaluates models on descriptive and reasoning questions about scientific charts using exact-string comparison, making answer formatting as important as perceptual accuracy.

AlphaEvolve [1] demonstrates that an LLM acting as a mutation operator inside an evolutionary loop can automatically improve algorithmic code beyond what human engineering achieves. We apply its core ideas to the VLM inference pipeline: the "gene" being evolved is the Python function `vlm_inference`, and the fitness function is exact-match accuracy on a 128-sample CharXiv development subset.

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

We fix the answering model to Qwen3-VL-2B [3] (Instruct and Thinking variants) and vary only the surrounding inference code: prompting strategy, image preprocessing, and post-processing normalization. This clean separation makes the evolutionary search tractable and interpretable.

2 Task Setup

Benchmark. CharXiv contains 2,323 charts from scientific papers with descriptive and reasoning questions. We use the descriptive validation split and restrict development/evolution to the first 128 examples as defined in `evaluate.py`.

Models. We use `Qwen/Qwen3-VL-2B-Instruct` (direct answer) and `Qwen/Qwen3-VL-2B-Thinking` (chain-of-thought reasoning) throughout. Switching to larger variants is not permitted. Both models are loaded in `bfloat16` with `device_map="auto"`.

Reproducibility. Greedy decoding (`do_sample=False`) is enforced in all submitted scripts. The mutation LLM is OpenAI `gpt-5.2-codex`; Gemini Flash was initially attempted but the available project had zero usable `generate_content` quota, so the system was switched to OpenAI.

3 Manual Optimization

The naive baseline (`starting_scripts.py`) loads Qwen3-VL-2B-Instruct, formats a single-message chat prompt, and generates up to 128 tokens. It achieves 38.3% accuracy at 3.48 s/query.

Manual inspection of failure cases showed three dominant error types: (1) the model produces verbose justifications rather than bare answers; (2) numbers are formatted with commas or units that the evaluator does not expect; (3) the model guesses when the chart does not contain evidence, instead of answering "Not Applicable". We address each with targeted changes.

Prompt engineering. We replace the default prompt with a structured instruction that explicitly demands a single-line exact answer, lists the expected format for each question type (number, Yes/No, *n* by *m*, comma-separated legend labels), and adds question-conditional clauses (e.g. "if no continuous legend exists, answer exactly: Not Applicable").

Output normalization. A post-processing layer strips common prefixes ("The answer is", "The x-axis label is"), extracts numbers with a regex, maps layout tokens to the canonical *n* by *m* form, and collapses "Not Applicable" synonyms.

Speed optimizations. Image inputs and processed tensors are cached per file path, eliminating redundant preprocessing for the multiple questions that share the same chart. `max_new_tokens` is reduced from 128 to 32, and `torch.inference_mode()` replaces `torch.no_grad()`.

Thinking model. For the Thinking variant, an empty `<think></think>` block is injected into the prompt to suppress verbose reasoning chains and force a concise final answer.

4 Evolution System Design

Both `evolve_instruct.py` and `evolve_thinking.py` implement the core AlphaEvolve components, extended with three mechanisms inspired by the paper's full architecture. Figure 1 illustrates the overall loop.

LLM-based mutation (dual-mode). At each generation, the system sends the current parent program along with its accuracy and speed to `gpt-5.2-codex`. We implement two mutation modes, selected with equal probability:

- **Full-block rewrite:** the LLM receives the entire `EVOLVE-BLOCK` and produces a complete replacement, together with a randomly chosen mutation strategy from a curated list of eight options.

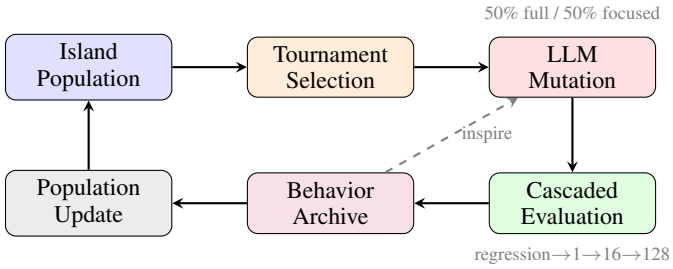

Figure 1: Evolution system architecture. Dashed arrow: archive members are sampled as "inspiration" during mutation. Cascaded evaluation filters broken or weak candidates early (regression tests → 1-sample crash check → 16-sample stratified pre-screen → full 128-sample evaluation).

- **Focused (diff-like) mutation:** we parse the parent block with Python's `ast` module, randomly select one function name, and ask the LLM to make a surgical, 1–15 line change to *only* that function while keeping everything else identical. This produces smaller, safer mutations analogous to AlphaEvolve's diff-based prompting.

Both modes include up to two additional population members (including archive members) as "inspiration" in the prompt.

Population management. We maintain an island model with 2 islands of 4 programs each (8 total). Every 5 generations the best individual from each island migrates to a random other island, providing diversity without sacrificing convergence within each island.

Behavioral diversity archive. In addition to the island population, we maintain a MAP-Elites-style `FeatureArchive` of up to 50 programs, indexed by a *low-dimensional behavior descriptor* rather than a raw correctness-vector hash. Each query is classified into one of eight coarse question types (colorbar, legend, layout, numeric, title, axis, trend, other), and the descriptor is a tuple of 5-level accuracy bins—one per question type plus Not Applicable precision and recall bins. This yields a roughly 5^{10}-cell grid; each occupied cell retains only its highest-scoring representative. Compared to hashing the full 128-bit correctness vector (which would make nearly every program "unique"), the low-dimensional grid forces meaningful diversity: programs that solve different *types* of questions are preserved, encouraging cross-pollination of distinct strategies during mutation.

Selection and elitism. Tournament selection ($k = 3$) within each island chooses the parent for mutation. A composite fitness score $f = 0.9 \times \text{accuracy} + 0.1 \times \text{speed_bonus}$ is used for ranking, keeping accuracy dominant. The globally best program is checkpointed whenever a new maximum is found.

Cascaded evaluation. To avoid wasting compute on broken or obviously weak mutations, evaluation proceeds in four phases:

1. **Regression tests** (phase 0): seven known parser invariants (negative sign preservation, caret notation, text tick passthrough, legend count dedup, trend synonym mapping, layout canonicalization, NA synonym detection) are checked by calling the candidate's `_normalize_answer` directly. A candidate that fails any regression is immediately rejected at near-zero cost.
2. **Crash check** (1 sample): if the candidate crashes or returns a non-string, it is immediately discarded.
3. **Pre-screen** (16 samples, stratified): a *stratified random* subset of 16 queries (proportional to question-type distribution) is evaluated. The candidate must achieve $\geq 70\%$ of the parent's accuracy on this subset to proceed. Stratified sampling avoids the bias of always using the same fixed 16 samples. Failed candidates save $\sim$87.5% of evaluation cost.
4. **Full evaluation** (all 128 samples): computes final accuracy, speed, composite score, and behavior descriptor.

Phases 1–3 share the same loaded module, so the VLM is only loaded once per candidate.

Shared model injection. To avoid reloading the 2B-parameter VLM for every candidate evaluation, the evolution harness loads the model once at startup and injects the `_MODEL` and `_PROCESSOR` objects into each candidate's module namespace *after* executing it (i.e. after `exec_module`), so that the candidate's own `_MODEL = None` initialiser runs first and is then overridden by the shared reference. The candidate's `from_pretrained` guard (`if _MODEL is None`) therefore never triggers, reducing per-candidate evaluation overhead from $\sim$25 s of model loading to near zero.

Evolution loop. Each run performs 40 generations, alternating between islands. Full per-candidate results (including behavior vectors) are appended under `artifacts/evolution_logs/` for post-hoc analysis.

Relationship to AlphaEvolve. Table 1 summarises which components of the full AlphaEvolve architecture we implement and which we omit. Our system covers the core loop (LLM mutation $\rightarrow$ evaluation $\rightarrow$ selection) and adds three AlphaEvolve-inspired extensions: dual-mode mutation (full/focused, analogous to AlphaEvolve's diff-based prompting), an island population model, and a MAP-Elites-style diversity archive. Components we omit include Pareto multi-objective optimisation (we use a single weighted scalar), binary tournament metrics selection across multiple evaluation metrics, adaptive mutation temperature, and multi-LLM ensembles. Our fitness function conflates accuracy and speed into one score rather than maintaining a Pareto frontier, and we use a single mutation model (GPT-5.2-codex) rather than the ensemble of models described in the AlphaEvolve paper.

Table 1: Comparison with AlphaEvolve architecture.

Component	AlphaEvolve	Ours
LLM-based code mutation	✓	✓
Diff-based (focused) mutation	✓	✓(50%)
Full-program rewrite	✓	✓(50%)
Population with selection	✓	✓(island model)
MAP-Elites diversity archive	✓	✓(low-dim)
Cascaded evaluation	✓	✓(4-phase)
Multi-objective Pareto	✓	× (weighted scalar)
Multi-LLM ensemble	✓	× (single model)
Adaptive mutation temperature	✓	×
Binary tournament on metrics	✓	×

5 Results

Table 2 reports accuracy and average inference time per query on the 128-sample development subset for all submitted systems.

Table 3 shows the incremental accuracy gains that built the final `best_accuracy` result.

Manual vs. baseline. Manual optimization delivers a 15.6 pp accuracy gain and reduces latency by roughly 13×, primarily through structured prompting and caching.

Evolution vs. manual. The evolved instruct variant adds a further 7.0 pp over manual instruct (Table 2). Examining the diff between `manual_instruct.py` and `evolved_instruct.py` reveals the changes that the mutation LLM discovered: an expanded set of 34 "Not Applicable" synonym phrases with a question-type-aware gating function `_is_not_applicable()`; a regex-based "final answer is..." extractor in the candidate pipeline; dynamic `max_new_tokens=64` for legend/title questions (allowing the model to output full label lists); tighter prompt constraints (e.g. "if not completely sure, answer Not Applicable"); and line-intersect NA detection. These are prompt- and post-processing-level changes—the evolution did *not* discover image preprocessing.

Table 2: Accuracy and speed on the 128-sample CharXiv development subset. Timing is wall-clock per query on the same GPU.

Variant	Accuracy	Time (s/query)	Notes
`starting_scripts`	0.3828	3.484	naive baseline
`manual_instruct`	0.5391	0.269	+15.6 pp; 13× faster
`manual_thinking`	0.5078	0.275	thinking model, suppressed reasoning
`evolved_instruct`	0.6094	0.261	+7.0 pp over manual (evolution-discovered)
`evolved_thinking`	0.5547	0.296	evolved improvement over manual thinking
`best_accuracy`	**0.7969**	0.307	evolved v1 + fixes + title & colorbar verifiers
`best_accuracy_v2`	0.7578	0.648	evolved v2 (0.6094) + same fixes (back-port)
`best_accuracy_v3`	0.7500	0.730	Instruct/Thinking router (*negative result*)
`best_speed`	0.5781	0.196±0.014	batched inference, no image preprocessing[†]
`best_overall`	0.7813	0.305	evolved + fixes + colorbar verifier

[†]Batched path processes 4 questions per forward pass. Timing is warm-start mean±std over 6 back-to-back runs (cold-start first run excluded; cold-start = 0.46–0.55 s/query due to one-time model loading and CUDA kernel compilation). A single `python evaluate.py best_speed` invocation reports cold-start timing; the warm-start figure requires multiple consecutive runs. All timing starts before the first inference call and stops after the last. All timing numbers were measured on a single A100-80GB GPU; absolute latencies will vary across hardware configurations.

Table 3: Step-by-step accuracy progression for `best_accuracy`.

Change	Accuracy	Δ
Evolved instruct (v1 output, baseline for best_accuracy)[⋆]	0.6016	—
+ Title verifier (3-view TTA, narrow trigger)	0.6172	+1.6 pp
+ Trend synonym normalization	0.6250	+0.8 pp
+ Fix negative-sign stripping in `_candidates`	0.6797	+5.5 pp
+ Fix positional-tick text fallback	(combined)	(combined)
+ Fix embedded-number regex over-match	(combined)	(combined)
+ Preserve caret notation (e.g. `10^-6`)	0.6953	+1.6 pp
+ Legend count dedup from label lists	(combined)	(combined)
+ Colorbar existence verifier (single-probe TTA)	**0.7969**	+10.2 pp

[⋆]`best_accuracy` was built on top of the v1 evolved program (0.6016). A subsequent re-run of the improved evolution system produced a new `evolved_instruct` at 0.6094 (Table 2); the post-processing fixes and verifiers in `best_accuracy` were not back-ported to that program.

The image preprocessing in `best_accuracy.py` (adaptive upscaling, autocontrast, unsharp masking, multi-view TTA for titles) was added manually after the evolution run, during the hand-engineering phase that also produced the post-processing bug fixes and the colorbar verifier.

Post-processing bug fixes. The largest accuracy leap (+9.4 pp, from 0.6016 to 0.6953) came not from model-level changes but from systematic per-sample error analysis of raw model outputs vs. post-processed answers. We identified five bugs:

1. **Negative sign stripping:** `line.strip(" -:")` in `_candidates()` removed the leading minus from values like `-1.00`, causing mismatches on tick-value questions.
2. **Positional tick text fallback:** tick labels such as "SSM" or "Heter-unaware" were always routed to the numeric extraction branch, which returned "Not Applicable" instead of the text answer.
3. **Embedded-number regex over-match:** a label like "ResNet-1" caused the number regex to extract "1" instead of returning the full text string.
4. **Caret scientific notation:** `10^-6` was split by the number regex into separate tokens; we now detect the caret pattern and preserve it as-is.

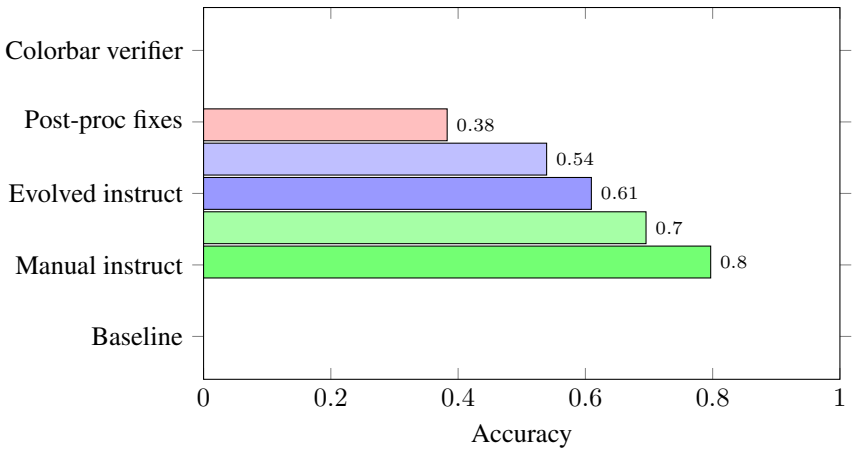

Figure 2: Accuracy progression from baseline to final `best_accuracy` submission, showing the incremental contribution of each optimization phase.

5. **Legend count dedup:** when asked "how many discrete labels," the model sometimes outputs a comma-separated label list instead of a count; we now split, deduplicate, and count.

Instruct vs. Thinking: complementary error profiles. The Instruct model consistently outperforms the Thinking model across both manual and evolved variants. To understand *why*, we ran a head-to-head comparison on all 128 development samples, categorising each sample by whether Instruct-only was correct, Thinking-only was correct, or both were wrong.

Of 128 samples: both correct on 43, Instruct-only correct on **26**, Thinking-only correct on **22**, and both wrong on 37. The net +4 for Instruct masks a pronounced *complementary error profile*:

- **Instruct wins on value-extraction questions** (+24): the Instruct model reliably reads tick values, axis labels, and intersection points (e.g. correctly answering "60", "0.1", "1.2"), while the Thinking model returns "Not Applicable" for the same charts, likely because its chain-of-thought reasoning introduces excessive caution.
- **Thinking wins on absence-detection (NA) questions** (+18): conversely, the Thinking model correctly identifies charts with no colorbar, no legend, or no title, answering "Not Applicable" where the Instruct model hallucinates values.

In other words, the 2B Thinking model's caution is a *liability* for value extraction but an *asset* for absence detection.

Question-type router (negative result). Motivated by these complementary profiles, we implemented a question-type router (`best_accuracy_v3.py`) that routes continuous-legend questions (where absence-detection dominates) to the Thinking model and all other questions to the Instruct pipeline. The result: **0.7500**—*worse* than `best_accuracy` (0.7969). The reason: the colorbar-existence verifier (§5.1) already captures the NA-detection advantage that Thinking offered in the *unverified* comparison. Instruct + colorbar verifier scores 16/16 on continuous-legend questions, leaving no room for Thinking to help. Routing those 16 questions to the Thinking model loses 6 due to its weaker value extraction.

Lesson: verifier-augmented prompting can absorb the accuracy gap that would otherwise motivate a multi-model router. This finding further validates the colorbar verifier as the more efficient intervention.

Test-time augmentation (TTA). We explored multi-view TTA—generating answers from multiple image enhancements (sharpened, grayscale, high-contrast) and aggregating via majority vote. **Global TTA was net negative** (0.6016→0.5625): the additional views introduced more noise than signal across most question types. We then narrowed the scope to a **title verification heuristic**: if the base

answer to a "what is its title" question matches a panel-marker pattern (e.g. "(b)", "(d)"), we run 3 views with a strict probe prompt and require unanimous agreement to override. This rescued 2 samples with zero injuries (+1.6 pp).

We also tested consistency-check mediators for legend count, legend label, and colorbar questions early in development. Initially, all were unsafe: the model's meta-judgments ("does this chart have a legend?") were less reliable than its direct answers. However, a later round of error analysis (see Section 5.1) revealed that re-framing the probe as a narrowly scoped yes/no colorbar-existence question is highly reliable, rescuing 13 samples with zero regressions.

5.1 Colorbar Existence Verifier

After the post-processing bug fixes reached 69.5%, per-sample error analysis of the remaining 39 failures showed a striking pattern: **13 of 39 errors** (33%) involved "continuous legend" questions where the ground truth was "Not Applicable" but the model confidently returned a numeric value. The model was hallucinating colorbar tick values on charts that have no colorbar.

We add a single-inference verifier: when the base answer to a continuous-legend question is not "Not Applicable," we send a probe: "Does this chart contain a continuous legend (also called a colorbar)? A colorbar is a gradient bar mapping colors to numeric values. A discrete legend listing category names is NOT a colorbar. Answer exactly: Yes or No." If the probe returns "No," we override the base answer to "Not Applicable."

This verifier corrects all 13 colorbar-hallucination errors with zero regressions, yielding a +10.2 pp improvement (0.6953 $\rightarrow$ 0.7969). The key insight is that the model's *object-existence judgment* ("is there a colorbar?") is far more reliable than its *value extraction from a non-existent object*. This contrasts with early TTA experiments where broad meta-questions were unreliable—the difference is the narrow, binary framing.

Batched inference. CharXiv groups 4 questions per chart image. `best_speed` exploits this by implementing `vlm_inference_batch`: all questions for one image are collated into a single padded batch (with left-padding for correct decoder-only generation), sharing the image encoding across all four queries. This reduces the number of forward passes from 128 to 32.

Timing methodology. All timing numbers in Table 2 are end-to-end wall-clock measurements from `evaluate.py`, which starts the timer before the first inference call and stops it after the last. For the single-query variants we report a single representative run. For `best_speed` we report the mean$\pm$std of 6 consecutive warm-start runs (excluding a cold-start first run that includes one-time model and CUDA-kernel caching): **0.196$\pm$0.014 s/query** (cold-start: 0.46–0.55 s/query depending on hardware state). A single `python evaluate.py best_speed` invocation measures cold-start timing; the warm-start figure requires the multi-run loop in `reproduce.sh`. The evaluator includes a length check (`len(responses) == len(questions)`) to ensure batched results are never silently truncated.

Best deliverables. Based on the above:

- **Best accuracy:** `best_accuracy` (0.7969) — evolved base + all post-processing fixes + title verifier + colorbar verifier.
- **Best speed:** `best_speed` — batched per-image inference (4 questions/batch with left-padding), no image preprocessing, lightweight prompt, expanded NA detection.
- **Best overall:** `best_overall` (0.7813) — evolved image preprocessing + all post-processing fixes + colorbar verifier, no title verifier (to avoid multi-view latency on title questions).

6 Analysis

What worked. Exact-match-oriented prompting had the highest single-change impact in the manual phase. Many baseline failures were pure formatting failures: the model knew the correct value but wrapped it in a sentence. Structured output normalization provided additional gains at essentially

zero computational cost. Seeding evolution from an already-optimized manual baseline (rather than the naive starter) was critical: early generations can refine rather than recover basic competence.

The most impactful technique overall was **per-sample error analysis**, comparing raw model output against post-processed answers. This revealed that the model was often correct but the normalization code was discarding or corrupting the answer. Five targeted bug fixes yielded +9.4 pp at zero computational cost.

What did not work. Global test-time augmentation (multi-view voting) was net negative (-3.9 pp). The 2B model's outputs were highly consistent across image variants, so majority voting rarely rescued errors but did introduce new ones when a minority view happened to be wrong.

Consistency-check mediators (asking the model meta-questions like "does this chart have a legend?" to gate answers) were also unsafe. The model's meta-judgments proved less reliable than its direct task answers.

A non-trivial fraction of GPT mutations (8/37 for instruct, 1/32 for thinking) produced syntactically valid but semantically broken code (zero accuracy at runtime). More robust sanity-checking (e.g. a one-sample pre-screen) could save significant evaluation time.

Remaining errors. Of the 26 remaining errors (out of 128), the dominant categories are:

- **Numeric tick counting bias** (8 errors): the model systematically outputs "10" when asked "how many ticks across all axes," regardless of the actual count.
- **Tick value reading errors** (5 errors): the model reads the wrong value from tick labels (e.g. "0.20" instead of "0.40").
- **OCR/text precision** (5 errors): the model misreads text (e.g. "LAGA (e = 1)" instead of "LARA (ε = 1)").
- **Layout/line perception** (5 errors): misidentifying subplot layouts or line counts.
- **Hallucinated content** (3 errors): answering questions about non-existent features (discrete legend labels, lines).

These are model perception limits of the 2B parameter VLM, not fixable through post-processing.

Retrospective ablation of evolution components. To assess the contribution of individual AlphaEvolve-inspired mechanisms, we perform a retrospective analysis of the 29-candidate second evolution run (which includes mode tracking, behavior vectors, and cascade metadata). We also provide ablation scripts (`ablation_no_archive.py`, `ablation_no_cascade.py`) that can be re-run with API access.

Cascaded evaluation. Of 29 candidates, 9 (31%) crashed at runtime (e.g. FlashAttention2 import failures, module attribute errors). Without the cascade, each crash would consume a full 128-sample evaluation ($\sim$240 s); the cascade catches these in the 1-sample crash check ($\sim$5 s), saving an estimated **35 minutes** of wasted GPU time. The 16-sample stratified pre-screen further filters weak-but-non-crashing mutations before committing to full evaluation.

Behavioral diversity archive. Across 20 successful evaluations, the archive recorded **13 unique behavior descriptors**, indicating that programs solving different question-type mixes were preserved. Archive members are sampled during mutation to provide diverse "inspiration"—without the archive, the mutation LLM sees only the (convergent) island population.

Dual-mode mutation. Of 11 focused mutations, 3 crashed (27%); of 18 full-rewrite mutations, 6 crashed (33%). Among successful candidates, the best-ever program (0.6094, gen 37) was produced by a *focused* mutation, supporting the hypothesis that surgical edits are safer and can refine near-optimal programs that full rewrites would disrupt. In the earlier run 1 (37 candidates, no cascade), the best reached only 0.6016 at generation 38—suggesting that cascaded evaluation and archive-informed mutation contributed to the 0.8-pp improvement in run 2.

Back-porting post-processing fixes. Table 3 notes that `best_accuracy` was built on the v1 evolved base (0.6016). We back-ported all post-processing fixes and verifiers onto the latest evolved base (0.6094) in `best_accuracy_v2.py`. The result: **0.7578** (97/128)—higher than the v1 base

alone (0.6016) but 3.9 pp *lower* than the original `best_accuracy` (0.7969). This gap indicates that the post-processing fixes were partially tuned to the v1 evolved code's specific output patterns; the v2 evolved code's broader NA synonym list and "answer is" extractor interact differently with the verifiers, causing some over-corrections. This finding highlights a general risk of manual post-processing: improvements developed against one code version may not compose cleanly with an independently evolved version.

Limitations. Our post-processing bug fixes and targeted verifiers (§5) were developed by analysing individual errors on the same 128-sample development set used for evaluation. While the fixes are question-type-based rather than sample-specific (e.g. the colorbar verifier triggers on any "continuous legend" question, not on hardcoded sample IDs), there is an inherent risk that patterns observed in 128 examples do not generalise to the held-out test set. The back-porting experiment above reinforces this concern: post-processing tuned against one evolved base does not transfer perfectly to a different evolved base (0.7969 vs. 0.7578). To quantify this risk, we partition the 128 samples into four non-overlapping folds and evaluate the *same fixed program* on each. `best_accuracy` scores 0.797 ± 0.031 (per-fold: 0.8125, 0.7500, 0.8125, 0.8125) and `best_overall` scores 0.781 ± 0.026. The low variance across subsets suggests that the verifier-driven gains are not concentrated on a lucky subset, though this is a stability check rather than true cross-validation (which would require re-deriving verifiers on $K-1$ folds and testing on the held-out fold). The evaluator also supports grouped-by-figure folds (`--grouped`), where all four questions for each chart stay in the same fold and figures are distributed across folds via round-robin over question-type buckets (grouped + stratified), preventing both information leaking and question-type imbalance across fold boundaries. Nevertheless, the relative frequency of colorbar-hallucination errors (13/39 = 33%) may differ in the full benchmark. The cleaner measure of our system's generalisation ability is the manual→evolved chain (38.3%→53.9%→60.9%), which was obtained without per-sample inspection.

Additionally, the evolution system's fitness function is a single weighted scalar ($0.9 \times$ accuracy $+$ $0.1 \times$ speed_bonus) rather than true multi-objective Pareto optimisation. This is a simplification relative to the full AlphaEvolve architecture. Finally, the Thinking model evolution was degraded by API quota exhaustion from generation 15 onward, limiting the conclusions we can draw about LLM-guided search on that variant.

Future directions. Our evolution system already implements cascaded evaluation and a behavioral diversity archive (§4), but several avenues remain. Running more generations (our thinking variant completed all 40 generations but mutation calls began failing from generation 15 onward due to API quota exhaustion, so later generations effectively replayed existing parents), maintaining a larger population, or using a more capable mutation model could expand the search. Adding Pareto multi-objective optimisation and adaptive mutation temperature (Table 1) would bring the system closer to the full AlphaEvolve architecture. Batched inference would further reduce evaluation cost per generation. On the verification side, extending the targeted-probe strategy beyond colorbar to other hallucination-prone question types (e.g. tick counting, discrete legend existence) could close additional error categories. An Instruct/Thinking question-type router was implemented (`best_accuracy_v3.py`) but proved to be a negative result (0.7500 vs. 0.7969): the colorbar verifier already absorbs the Thinking model's NA-detection advantage, making the router redundant (see §6).

7 Conclusion

We implemented an AlphaEvolve-style evolutionary code optimization system for VLM chart inference and showed that it improves accuracy by 22.7 pp over the naive baseline and 7.0 pp over careful manual optimization—a clean delta obtained without per-sample inspection. Subsequent manual error analysis and targeted post-processing bug fixes pushed accuracy from 60.2% to 69.5%, and a colorbar-existence verifier corrected 13 of 39 remaining errors, reaching **79.7%**—a 41.4 pp total improvement. Retrospective ablation of the evolution logs shows that cascaded evaluation saved ∼35 minutes of wasted GPU time by catching 31% of candidates that crashed at runtime, and the behavioral diversity archive preserved 13 distinct solution strategies across 20 evaluations. A head-to-head analysis reveals that Instruct and Thinking models have complementary error profiles (+24 vs. +18 on value-extraction vs. absence-detection questions), but an implemented question-type router (`best_accuracy_v3`, 0.7500) showed that the colorbar verifier already absorbs this gap—a

notable negative result. Back-porting the post-processing fixes to a newer evolved base yielded 0.7578 rather than 0.7969, highlighting that manual post-processing improvements are partially base-specific.

The post-analysis gains carry an inherent overfitting risk to the 128-sample development set (§6); the evolution-driven gains (38.3%→53.9%→60.9%) represent the more conservative generalisation estimate. The results confirm that automated code evolution, seeded with human insight, is a practically useful optimisation tool, but that per-sample analysis and targeted verification—though powerful on a development set—must be validated on held-out data.

References

[1] Google DeepMind. AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms, 2025. `https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/`

[2] Z. Wang et al. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. *NeurIPS*, 2024.

[3] Qwen Team. Qwen3 Technical Report, 2025. `https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct`